Terminal Bench Reviewer Guidelines

👋Welcome to **TerminalBench**! Many of you may have been part of **Selection Improvement**, which was the precursor to this project. Your task will be to **review** attempted tasks and **generate files within Outlier** from successful attempts.

What Makes a TB Task

- It is a real professional task in your domain that requires a computer to complete.
- 💻It can be solved entirely from a **Linux terminal** (via Python scripts, shell commands, or other CLI tools)
- It has a single, objective, verifiable answer with a well-specified output format.
- 🏅It is hard enough that frontier AI models cannot reliably solve it.
- A qualified expert who knows the correct approach can implement the solution in a few hours to one working day.
- All required data comes from sources without usage restrictions (open-source, public domain, or hand-crafted).

📖**Table of Contents**

Review Workflow
- 1. Check the Prompt
- 2. Check the Golden Solution
- 3. Check the Verifiers
- 4. Check the Evaluation
- 5. Check Difficulty

Decision Standard
Common Blocking Issues
Reviewer Checklist
Reviewer Output Quality

Change Log

Date	Section	Change
Oct 19, 2026	**Updates on difficulty / Run Agent Trials**	- Models updated in run-trials to: GPT 6.1 Sol, GPT 6 Sol, and Grok 4.7 (all highest reasoning). - GPT 6 Sol replaces Gemini as our difficulty proxy from L0 to L1; it must fail at least 2 of its 3 trials.

CONFIDENTIAL | arjunguguloth03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

		- Difficulty bar otherwise unchanged: a minimum of 5 of 9 trials must still fail. - Clarified that agent trials are required for every L0 sandbox session before a task can be sent forward (Testing Summary row 4 corrected from "optional" to "required"). - Model Stump at L-1 is now GPT 6 Sol; a failing response is still required. - Added a note to refresh the sandbox to pick up updated models.
Oct 8, 2026	**Input File License**	- Updated to include guidance for new third party source fields
Jul 24, 2026 Sep 30, 2026	**Updates on difficulty**	- several sections updated regarding a minimum of ⅔ failing Gemini trials for a task meet our difficulty bar in addition to a majority of all trials failing (5/9) at least. - models have been updated in run-trials to be: Gemini 3.8 flash, GPT 5.6 Terra, and Grok 4.5 (all highest reasoning)
Jul 21, 2026	**Run Insight Checks**	- Added /run-insights as part of the sandbox workflow
Jul 16, 2026	**Reviewer Workflow Summary**	Added two flow charts that summarize the review process of reviewing a task from scratch
Jul 14, 2026	**Run Agent Trials**	-/run-trials is now required (overall time has reduced)
Jul 1, 2026	**Input File License** **FAQ**	-Added note on copyright requirements -If not SBQing task, and changes needed once you get to sandbox, you can make changes in sandbox itself (no need to go back in task and regenerate responses)
Jun 25, 2026	**FAQ**	-Added claim / pause time
Jun 24, 2026		Entire Doc Update

Review Task Overview

The reviewer's role is to ensure each task is **high-quality, rigorous, difficult**, and likely to pass evaluation — not just that it appears well-written.

Reviewers should evaluate tasks against the 7 core criteria:

- **Verifiable:** The task must be deterministically checkable with a program that will always detect errors and always return "valid" for correct solutions.
- **Well Specified**: Nothing is left up to guessing or can be interpreted differently. 2 experts solving correctly will arrive at the same answer.
- **Solvable:** The problem must have a solution, or must have a convincing argument the problem is solvable. The solution itself should be implementable in a few hours by someone who knows exactly what to do.
- **Difficult:** Requires PhD-level knowledge or years of professional domain expertise. The task must stump the best of the best AI models.
- **Realistic, valuable, and interesting**: There should be some real-world scenario where solving this problem has marginal utility. Someone could get paid in the real world to do this.
- **Outcome-Verified**: Graded on the final result, not the approach. The agent chooses its own path to reach the outcome.
- **Requires Code:** the task must be able to be solved entirely from a Linux terminal (via Python scripts, shell commands, or other CLI tools)

✏️ *A task should only be accepted if it is consistently strong across all seven.*

You are not judging the attempter's intent. You are judging whether the task holds up under strict review.

Ask yourself:

- Could a model only reach the solution by doing the intended hard work?
- Could a deterministic script reliably verify the result?
- **Would an expert consider this genuinely difficult and realistic?**

❌ If the answer to any of these is no, the task is at risk.

#	Step Title	Description
1	**Review Prompt**	**Carefully read through the prompt to ensure it is difficult, requires domain expertise, is entirely solvable within a terminal / command line interface, and has a single deterministic, verifiable solution.** Also, in this step, review the LLM evaluations: 1. Model Stump a. **Ensure GPT 6 Sol, the model run on the final prompt, fails to provide a correct answer** and that the attempter's justification that the model failed makes sense and is aligned with the Golden Solution.

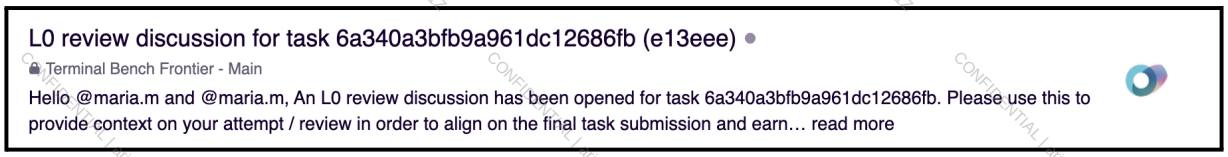
		FEATURE NOW LIVE: Upon you claiming a task to review, an Outlier Community Topic will be started with the attempter of the task for you all to discuss. This is very important to provide you full context on the task.
2	**Review Golden Solution**	**The golden solution should provide clear step-by-step instructions to reach the final answer.** This should cover every explicit ask from the prompt. This should be logical and thorough such that you (or a coder) can implement the reasoning for the solution.
3	**Review Verifiers**	The verifiers should cover ONLY explicit final answers from the prompt. They should not touch on any intermediate steps. **It should verify that the golden solution provided is 100% correct and any incorrect solution is wrong.**
4	**Review Overall Difficulty and Quality of Task**	After reviewing the full task (prompt, golden solution, verifiers), **determine overall quality of task,** according to project criteria specified above. Before submitting, make sure you are bringing any questions or clarifications to the Outlier Community Topic created with the attempter, if possible.
5	**Generate Files via Outlier Sandbox**	**If the task does not need to be sent back to the attempter for fixes (AKA NO SBQ),** continue to the Sandbox to generate files and upload to Outlier (detailed instructions below).
6	**Finalize Files**	Final checks will be run on the files after capturing them from the Sandbox. **Use the feedback from these to iterate on the files.**

Review Workflow

1. Review the Final Prompt

- **Read the final prompt,** its accompanying files, and the difficulty explanation (this is after the proposal section). Ensure the 2nd LLM (GPT 6 Sol) is stumped and does not solve the prompt.
    - ‼️ The prompt is the most important section because it is the only part the model sees (along with any attached and referenced files). The prompt must be aligned with all the criteria in the Task Overview.
- **Note:** Upon you claiming a task to review, an Outlier Community Topic will be started with the attempter of the task for you all to discuss. This is very important to provide you full context on the task.

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

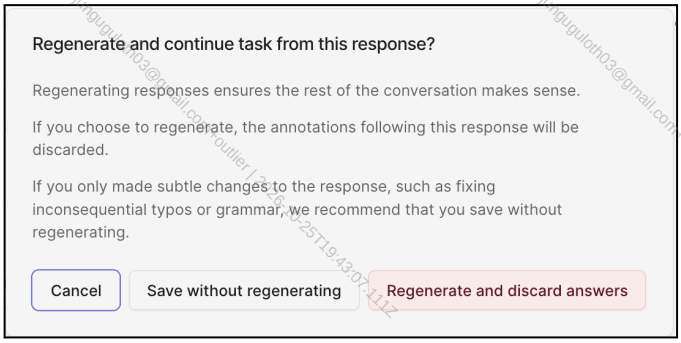


👀Reviewers should check:

Domain Expertise 👓	☐ Does the prompt describe a problem that requires **real domain expertise**? ☐ A strong task should require specialized knowledge, not just general programming ability.
Code or Computer Requirement 🧑‍💻	☐ Does the task *reasonably* require code or a computer to be answered by a human, and can it be **answered entirely from a terminal** (files, code, scripts, shell commands)? If you are unsure, feel free to ask an AI model to answer this question for you, given the prompt. ☐ 🛑**If the correct answer can be produced without writing or running code, or if the task requires ONLY text judgment/analysis the task should be rejected.**
Verifiable Deliverables ☑	Does the prompt request deliverables that can be checked and evaluated with an automated script? Is **the final answer** not open to interpretation? Check whether the prompt clearly specifies: • File names • File paths (using “/app/filename”) • File formatting • Required output contents If the output cannot be evaluated **deterministically** from the requested deliverables, this is a blocking issue (SBQ or FIX). If there are **multiple possible final answers**, this is a blocking issue (SBQ or FIX).
Ambiguity 🔖	Is the prompt unambiguous? A reviewer should ask whether two subject matter experts would reasonably agree on what a correct answer looks like. Some ambiguity may only become clear during the difficulty or verifier review, but clearly ambiguous prompts should be returned for revision. Avoid:

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

	• Vague success criteria • Missing output requirements • Overly step-by-step instructions - *we want to test the model's ability to reason through tasks without direct instruction* • Process-based grading instead of outcome-based grading
Difficulty 📝	*The most important aspect of the task is its difficulty!* • ✅Confirm the task stumped the model (GPT 6 Sol) ○ Does the attempter's stump explanation make sense and actually describe somewhere the model fell short? Does it align with the provided Golden Solution? • Compare the final prompt with the difficulty explanation to determine if they are consistent with each other. • Ask yourself - "Would this task require PhD-level (or experience-equivalent) knowledge and take a few hours or more for a human to complete?" If you think an undergraduate student could do it, the task is too easy.
Technical Phrasing 💻	• In addition to making edits for Code/Computer requirement, check that all parts of the prompt that reference files or other components within the directory are phrased in the proper technical way. **All files should be referenced as "/app/filename"**

> ‼️ **Note:** You may see this pop up:
>
> 
>
>
> ➔ Always click "*Save without regenerating*" or you will lose the attempter's answers.

❌ Common Prompt Errors:

Category C: Whole third-party files used as-is or modified

The contributor included someone else's dataset, code, document, image, font, or model file, whether unchanged, subset, or edited.

➔ Continue to Step 3 (content table.

Step 3: Does the source license permit reuse?

Check the source's license against the table below:

Content: Data, Documents, and Images

License	What it means	Usable?
CC0	"No rights reserved": fully public domain, do anything	✅ Best option
CC BY	Free to use; credit the author, link the license, note any changes	✅ Fine
CC BY-SA	Free to use with credit; any adapted version must also be CC BY-SA	✅ Fine
CC BY-ND	Free to use with credit, but no modifications of any kind (including cropping, subsetting or reformatting)	⚠️ Only if used exactly as published
CC BY-NC (and NC-SA, NC-ND)	Not for commercial use	❌ Reject: Scale AI work is commercial
All Rights Reserved / Getty / Shutterstock	Standard copyright, no reuse	❌ Reject
Unknown / Unclear	License information absent or ambiguous	➔ See Step 4

For code, also check that the license file sits with the code inside the zip file provided (e.g. vendor/LICENSE).

- If the license clearly permits reuse ➔ no issue; proceed.
- If the license clearly prohibits reuse ➔ go to Step 5.
- If the license cannot be determined ➔ go to Step 4.

Step 4: License cannot be determined

If you cannot verify whether the source license permits reuse (e.g., license information is absent, ambiguous, or the source is a paywalled publication with no visible license):

➔ Give the contributor the benefit of the doubt.

➔ Do not SBQ the task on this basis.

➔ Leave a note for the contributor asking them to clarify the source license or confirm how the material was used.

➔ If the contributor can confirm that only factual data values were extracted (not expression), no further action is needed.

Step 5: Copyright issue identified

If you have determined that the contributor reproduced protected expression from a source that does not permit it:

➔ Return the task to the author with a score of 2–3/5 and clear written feedback explaining the issue.

In your feedback, distinguish between the two situations:

Fixable: The material can be reformatted so that only the underlying data is retained (not the table structure, figure, or other protected expression). Recommend that the contributor:

- Re-extract the factual values into their own format (e.g., a plain CSV they create themselves)
- Remove the original file from the task
- Update the third-party answer and the "Third-party sources" field: keep the original source's link as a citation for the facts, or answer No if the files are now entirely their own work

Not fixable: The expression is integral to the task (e.g., the task requires the agent to work with the original figure or copyrighted document itself). Advise the contributor that the task needs to be redesigned around a freely licensed or self-created equivalent.

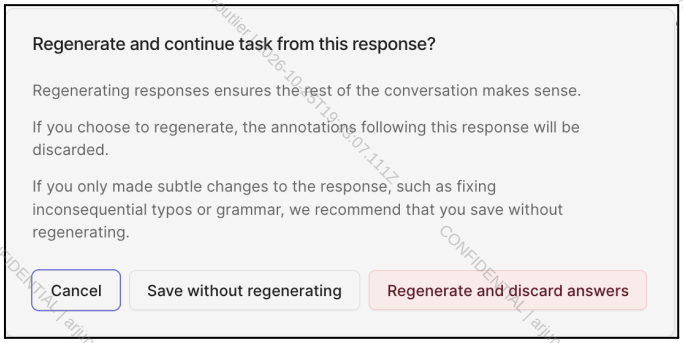
Quick Reference Summary

Situation	Action
CB used their own created work, declared in difficulty explanation field	No check needed
CB extracted facts/data only; own format; no expression reproduced; source cited	Acceptable; confirm source is cited
CB reproduced expression; source license permits reuse (CC0, CC BY, CC BY-SA)	No issue; proceed
CB reproduced expression; CC BY-ND source	Flag if file was modified or cropped
CB reproduced expression; license unclear or unverifiable	Benefit of the doubt; request clarification
CB reproduced expression; license clearly prohibits reuse (CC BY-NC, All Rights Reserved, etc.)	Return to author, score 2–3/5; explain issue; recommend fix if fixable

Source or file provenance not cited anywhere in the task	Request citation; do not SBQ alone

2. Check the Golden Solution

- **Try to go through the solution and solve the prompt** – this may be difficult and require conversation with the attempter (through Outlier Community Topic)
- Reviewers should check whether it is **complete enough to demonstrate solvability.** There is an LLM-generated solution breakdown. Feel free to edit this as needed if there are any necessary changes.
- For the LLM breakdown of the solution, **ensure the expert attempter answered or addressed all clarifications required.**
    - If they did not address required clarifications please reach out to them via the same Outlier Community Topic.
- Also, check the solution explanation and **make sure it matches the LLM output.**

> ‼️ **Note:** You may see this pop up:
>
> 
>
>
> ➜ Always click "*Save without regenerating" or you will lose the attempter's answers.*

Reviewers should check:

Step-by-Step Solution 🪜	☐ Does the golden solution provide a clear **step-by-step path** for solving the task? ☐ The solution should not simply state the final answer. It should **explain how the answer is reached**.
Intermediate Steps 👣	☐ Does the golden solution contain intermediate steps, calculations, implementation details, or reasoning?

	☐ A strong golden solution should give coders enough information to implement the solution correctly.
Alignment With Prompt ⚖️	Does the golden solution solve the exact task requested in the prompt? Red flags include: • Missing steps or logical gaps • Hidden assumptions that any *reasonable expert* would not know • Unrealistic implementation scope • Final answer only, with no supporting process • Mismatch between the prompt requirements and the solution
Difficulty Assessment 💭	Does the golden solution differ from what the model step outputted, and does the stump explanation align (repeated from above, but important to check)?
Explanation Alignment ⚖️	Read over the Final Solution Explanation and ensure its aligned with the LLM output of the solution.

3. Check the Verifiers

- The verification process is how the benchmark determines whether a solver's output is correct. **You are describing the logic of a deterministic verifier script** — a program that takes the agent's output files, checks them, and returns ✅PASS or ❌FAIL with zero human judgment.
- 🔑**The verifiers ONLY check the final output, not any intermediate steps.**
- For the LLM breakdown of the verifiers, **ensure the expert attempter answered or addressed all clarifications required.**
    - If they did not address required clarifications *and you agree that the clarification is necessary*, please reach out to them via the same Outlier Community Topic.
- The goal is to create an explanation that guides us to create that piece of code (the verifier script).
    - **Check the "Checks Extracted" section and make sure they would cover all edge cases of correct solutions and incorrect solutions.**
- Also, check the verifier explanation and **make sure it matches the LLM output.**

Reviewers should check:

Coverage 📋	Do the verifiers test **every explicit request** in the prompt? If the prompt asks for a specific file, format, value, or output, the verifier should account for it.
Determinism 🔍	Are all verifiers **deterministic**? ‼️ Verifiers should **not rely on subjective judgment or LLM-based evaluation**. They should not check methodology, internal reasoning, or algebraic expression form when multiple equivalent forms could be correct.
Output Scope 💻	Do the verifiers only inspect final outputs or files explicitly requested in the prompt? Verifiers **should not check hidden reasoning steps or intermediate artifacts** unless those artifacts are explicitly required as final deliverables in the prompt.
Test Specificity 🧪	Do the verifiers explain how to perform the test, not just what is being tested? **A verifier description should be specific enough that a coder can implement it as a repeatable script.** Key rule: If you cannot verify it with a script using only the requested outputs, it is not valid. Common verifier failures: • Checking files not required in the prompt • Validating process instead of final outcome • Using subjective or fuzzy checks • Missing validation for required outputs • Checking methodology or reasoning instead of output The same standards apply to golden solutions and verifiers proposed by an LLM and later fixed by the contributor.

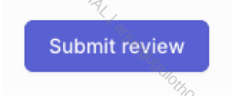
4. Overall Task Quality Judgement

❌ Spam

A spam task is considered something that clearly does not follow the guidelines, shows limited effort (under 30 min), and no potential for an attempter to improve. This should be used conservatively.

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

*If you mark the task as **spam** always click the following button to submit after the final feedback step:*



💬 Any Questions or Clarifications

➜ This is in **Outlier Community** 💬

Once you claim a task, a **Topic will be created on Outlier Community**. You and the attempter will be tagged.

Your job here:

- ☐ Ask any clarifying questions you may have!
- ☐ Clarify any edits being made to the LLM-generated Solution Breakdown and Verifier List
  - ☐ If the attempter is not responding, use your best judgement, especially if you are experienced within that domain ➜ do not wait for the attempter if they are not responding and SBQ as needed.

‼️ **Difficulty** is often the most misjudged part of review.

✅A task is difficult if it requires:

- Deep domain expertise
- Multi-step reasoning and implementation
- Non-trivial debugging, analysis, or system interaction
- Realistic professional judgment
- **AND STUMPS MULTIPLE FRONTIER MODELS**

❌A task is **NOT** difficult if it is:

- Merely long or tedious
- Based on obscure facts
- A standard textbook problem
- Solvable by a junior developer in a few days
- Hard only because of noise, scale, or unclear instructions

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

⚠️ **Important constraint:** Tasks should **not rely on lack of access to external information** to enforce difficulty. Instead, difficulty must come from reasoning, implementation complexity, and well-designed constraints that cannot be bypassed by simple lookup.

Watch for:

- Familiar patterns or standard workflows
- Hidden shortcuts (including the ability to look up answers from a publication from which the task is inspired)
- Superficial complexity (e.g., unnecessarily contrived constraints)
- Weak constraints that allow approximate or lookup-based answers
- Reasoning-only paths that avoid real code execution

If a task is well-structured but likely too easy in practice, it should not be accepted as-is.

💡 Additional Difficulty level tips (consider these when giving feedback to the attempter)

Tasks must be difficult enough that ***PHD-level (or experience-equivalent) knowledge*** is needed to solve.
➜ This means any task an average undergraduate could solve in under a few days is **too easy**.

- E.g. Course-project work, small compilers, basic OS components, textbook algorithms, are all too easy

❌ Things that **don't** add useful difficulty:

- Making the task **bigger** (extract 100 images instead of 10).
- Adding **parallel** deliverables (more clerical surface area).
- Throttling CPU/memory or tightening timeouts.
- Designing convoluted output schemas or formats. If GPT 6 Sol Terra fails your task because of a dollar sign in a float field, you are measuring format compliance, not capability.

✅Things that **do**:

- The agent has to *think before doing*. (e.g. Approach selection is non-obvious; pattern-matching to a familiar idiom gives the wrong answer)
- The right move requires **reading what's actually in the environment**, not assuming a standard layout.
- Something is broken and the agent has to **diagnose** *what* before fixing it. The oracle solution should itself look like an investigation.

- The task spans interdependent steps where an early wrong choice cascades and the agent has to recover.
- It targets the jagged edge of LLM capability, things humans find trivial but agents stall on (interactive TUIs, careful long-horizon state tracking, multi-turn coherence).

🏅 Decision Standard

Criteria	Strong Accept - 5	Accept - 4	Uncertain - 3	Reject - 2	Strong Reject - 1
Verifiable	Output is fully deterministic. A script can check it with zero human judgment. Tolerances are justified and tested against both correct and intentionally wrong answers.	Output is deterministic with minor formatting ambiguities that are unlikely to affect automated checking.	It's unclear whether all outputs can be checked programmatically, or tolerances seem arbitrary/untested.	Key outputs rely on subjective judgment, LLM-as-a-judge, or methodology checks rather than outcome checks.	No programmatic verification is possible; outputs are open-ended, subjective, or non-deterministic across runs.
Well Specified	Two independent domain experts would always arrive at the same answer. All file paths, formats, conventions, and parameters are explicit. Zero ambiguity.	Minor underspecification that a domain expert could resolve with one reasonable assumption. Would not change the final answer.	Ambiguity exists that could plausibly lead to different valid answers depending on interpretation. Needs clarification.	Multiple reasonable interpretations yield different answers. Missing output format, conventions, or key parameters.	Prompt is vague, contradictory, or so underspecified that no single correct answer can be identified.
Solvable	Golden solution provides complete, reproducible step-by-step reasoning. All files/data are provided and restriction-free. Implementable in a few hours by a knowledgeable expert.	Solution is convincing with minor gaps a skilled coder could fill. All required resources are present.	Solvability is plausible but not demonstrated — missing intermediate steps, unclear if resources are sufficient, or scope may exceed a working day.	Logical gaps, missing dependencies, unreproducible steps, or the solution requires resources/access not provided in the environment.	The task is unsolvable, infeasible within scope, requires external access not available, or the golden solution is fundamentally incorrect.
Difficult	Requires PhD-level or years-of-experience domain knowledge. Stumps frontier models due to genuine reasoning/expertise demands. Failure modes are coherence or verification failures, not environment issues.	Requires strong domain expertise. Model was stumped, though failure may partly stem from task structure rather than pure domain difficulty. Minor risk a strong agent could solve it.	Difficulty is unclear — could potentially be solved by a skilled undergrad or frontier model with a few attempts. Stump explanation is weak or doesn't align with golden solution.	Task is solvable by a competent developer in under an hour, or difficulty is artificial (tedium, obscure trivia, contrived constraints, convoluted output formats).	Standard textbook problem, course-project level, or trivially solvable by any junior developer or frontier model on first attempt.
Realistic, Valuable, and Interesting	Directly mirrors a task a domain professional gets paid to do. Clear real-world application with marginal utility. Data is realistic or real.	Rooted in a real scenario with minor abstraction. A professional would recognize it as relevant to their work.	Loosely connected to real work but feels contrived or academically artificial. Marginal utility is unclear.	Artificial construct with no clear real-world counterpart. No one would encounter or be paid to solve this.	Purely synthetic puzzle, trick question, or gimmick with zero professional relevance.
Outcome-verified	Graded entirely on final output.	Graded on output with minimal	Some verifier checks may	Verifiers check intermediate	Grading is fundamentally

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

	Agent has full latitude to choose methods. No process-based grading. No reward-hacking path exists.	process constraints that exist only to prevent cheating. No obvious reward-hacking path.	inadvertently constrain methodology, or a creative reward-hacking path might exist.	steps, specific tool usage, or methodology rather than outcomes. Or a clear reward-hacking shortcut exists.	process-based, or the answer can be trivially hardcoded/looked up to pass verification without doing the work.
Requires Code	CodeTask is impossible without writing and running code. Solution involves non-trivial scripting, computation, or system interaction entirely within a Linux terminal.	Task clearly requires code, though the prompt could make the computer requirement slightly more explicit.	It's ambiguous whether code is truly necessary, or parts of the task could be solved through reasoning or manual lookup alone.	Core answer can be produced without running code, or the task requires GUI tools, external services, or resources outside the terminal.	No code or computer is needed at all; the task is pure text analysis, reasoning, or subjective judgment.

Regardless of other models' performance, if GPT 6 Sol passes more than 1/3 of validated trials, this criterion cannot be rated Accept or Strong Accept — it defaults to Reject, and the task must be SBQ'd.

✍️Reviewer Feedback Quality

Feedback must be:

- Specific
- Actionable
- Tied to rubric criteria
- Clearly labeled as blocking or non-blocking
- Based on the task, not the *tasker*
- Respectful and professional, ***not*** **demeaning, rude, condescending, or dismissive**

Such that if the attempter was to implement all your written feedback without introducing new problems, the task would be perfect.

Example:

Blocking — Verifier Issue: The verifier checks an intermediate file that is not required in the prompt. Either the prompt must explicitly require this file as a deliverable, or the verifier must be rewritten.

❌ Common Blocking Issues

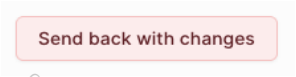
Reject or return for revision if:

- The task can be solved without code, or needs something outside of the terminal
- The outputs are not deterministically verifiable
- The prompt is ambiguous

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

- The verifier checks non-requested artifacts
- The golden solution is incomplete or inconsistent
- The difficulty is superficial
- The task relies on subjective judgment
- Required file names, paths, formats, or deliverables are missing
- The verifiers do not test every explicit request in the prompt
- The evaluation contains unresolved concerns

*If you mark the task as **Does this task still need fixes** = **Yes** always click the following button to submit after the final feedback step:*

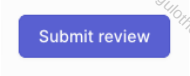


Reviewer Checklist

Before approving, confirm:

- ☑ The prompt requires domain expertise
- ☑ The task requires writing or running code
- ☑ The task cannot be solved by reasoning alone
- ☑ The prompt requests clear, verifiable deliverables
- ☑ File names, paths, and formatting are specified where needed
- ☑ The prompt is unambiguous
- ☑ The golden solution includes step-by-step guidance
- ☑ The golden solution includes intermediate steps, not just the final answer
- ☑ The verifiers test every explicit prompt requirement
- ☑ The verifiers are deterministic
- ☑ The verifiers explain how the test is performed
- ☑ The verifiers only check requested outputs
- ☑ Existing evaluations are Accept or Strong Accept, or any lower ratings are validly disputed
- ☑ The task is genuinely difficult, not superficially complex
- ☑ The task reflects a real-world scenario

*If you mark the task as **Does this task still need fixes** = **No** always click the following button to submit after the final feedback step:*



CONFIDENTIAL | arjungugulothi03@gmail.com+outlier | 2026-10-25T19:43:07.111Z

Continue to step 5 only if the task needs no more fixes (e.g., it is highly difficult, verifiable, unambiguous, correct solution...).

Otherwise the task will be SBQd back to the attempter.

If you find issues (e.g., difficulty) when using the Sandbox tests, you should copy the feedback to provide to the attempter and go back to the quality step, paste the feedback, add notes, and then change your decision to "Does this task still need fixes? = Yes".

PLEASE WATCH THIS WALKTHROUGH BEFORE CONTINUING!!!

5. Generating Files Using the Outlier Sandbox

In this step, you will take the task information already written in Outlier and convert it into the required file format using Claude Code inside the Outlier Sandbox.

> **Important:** Only complete this step if the task meets quality standards or you have already fixed the identified issues. If the task is fundamentally flawed, do **not** generate files. Instead, send the task back to the queue with clear feedback and submit your review.

‼️ 🛑 Found an issue while in the Sandbox? You must reverse your review.

If you already selected "Does this task still need fixes? = No" and started implementation, but then find a problem, **do all 5 steps before leaving**:

1. Copy your feedback first (changing the field erases your sandbox files).
2. Go back to the Quality step.
3. Change "Does this task still need fixes?" to Yes.
4. Update the score and paste the feedback.
5. Click "Send back with changes" (red button) — NOT "Submit Review" (purple button).

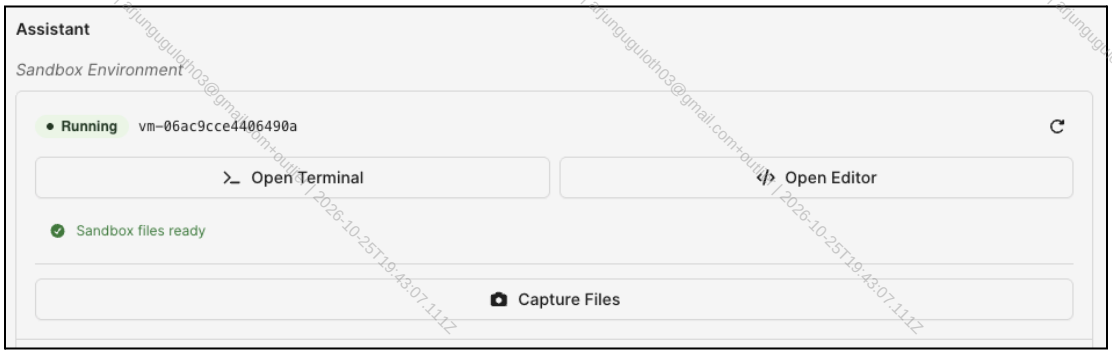
Using Claude Code in the Outlier Sandbox

Sandbox Setup

1. Click **Create Sandbox**.



2. Click **Open Editor**.



Note: If you see a **"Files failed to load"** error, click **Regenerate Sandbox** in the top-right corner, then reopen the editor.

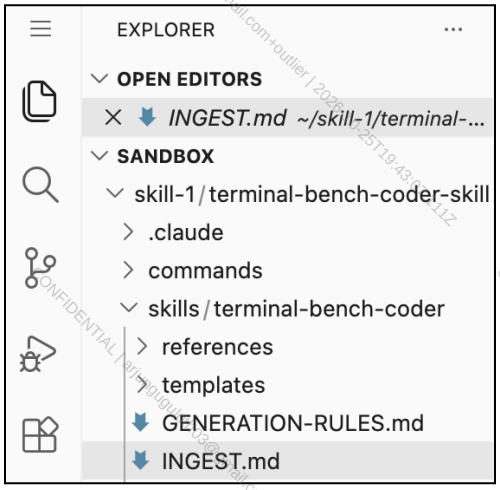


Verify the Ingest File

1. Open the file explorer.

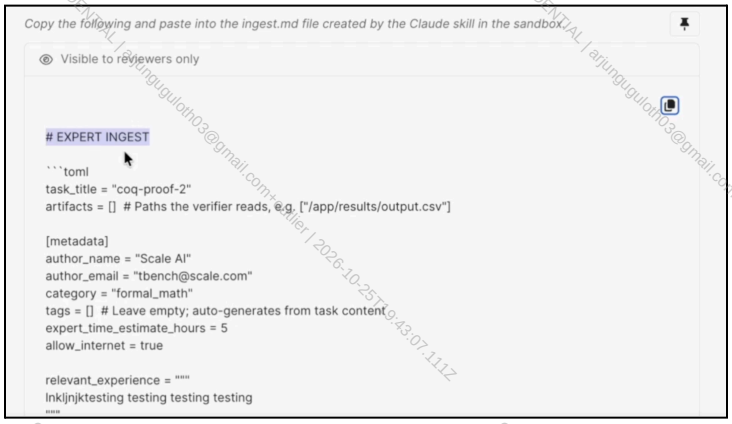
2. Navigate to:

```
None
sandbox/INGEST.md
```



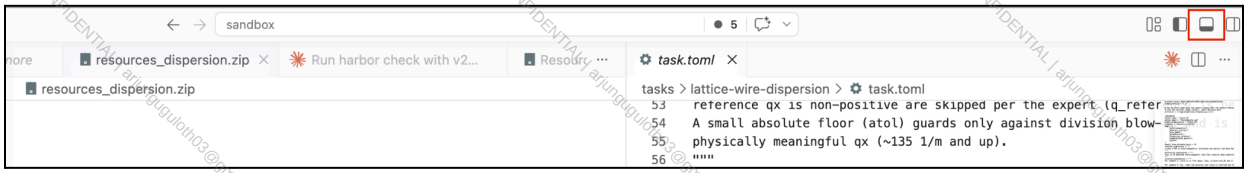
3. Verify that the contents match the information shown in the Outlier task under:

```
None
# EXPERT INGEST
```



Initialize Claude Code

1. Open a terminal using the terminal icon in the top-right corner of the editor.



2. Run:

```
Shell
claude
```

This initializes Claude Code for the workspace.

Build the Task Files

In the terminal, run:

```
Shell
/build-task
```

This command runs the full pipeline end to end:

```
None
generate files → oracle → nop → harbor check → verifier audits
```

Claude will automatically initialize the task, create the required files, validate them, and provide verifier feedback.

This command will also provide a quick difficulty check, is relatively fast but less accurate and may under-estimate task difficulty in comparison with the full agent trails.

⚠️ Running this multiple times will override the previous versions of files built — please use only the first time to build.

Expected Task Structure

The completed workflow should produce a task directory with the following structure:

```
None
tasks/<task-name>/      # 'tasks' folder is required
├── instruction.md      # Task instruction for the agent
├── task.toml           # Configuration and metadata
├── environment/
│   ├── Dockerfile      # Container environment setup
│   └── ...             # Optional: data files, such as files in data/
├── solution/
│   ├── solve.sh        # Reference solution / Oracle
│   └── ...             # Optional: scripts used by solve.sh
└── tests/
    ├── test.sh         # Test script
    └── ...             # Optional: pytest files used by test.sh
```

Note: The tasks/ folder is required in order to capture files successfully.

⚠️ **Please read through all generated files and ensure they are filled.**

If You Encounter Issues

1. **Timeouts:**
    a. **If Claude is timing out, please instruct it to continue (or switch to terminal instead of Claude Chatbot)**

If you run into other Sandbox or Claude-related issues, contact a QM in the War Room or OC.

If Claude feedback indicates problems with the original task design:

1. Copy the relevant Claude feedback.
2. Return to the Quality Review section.
3. Document the issue clearly.
4. Update your review score if needed.
5. Set **Fixes Needed** = **Yes** if appropriate.

🚀 Testing and Validation

Once your files have been generated, validate that all required checks have completed successfully. Higher-quality tasks that pass difficulty and validation checks on the first submission are more likely to qualify for quality incentives.

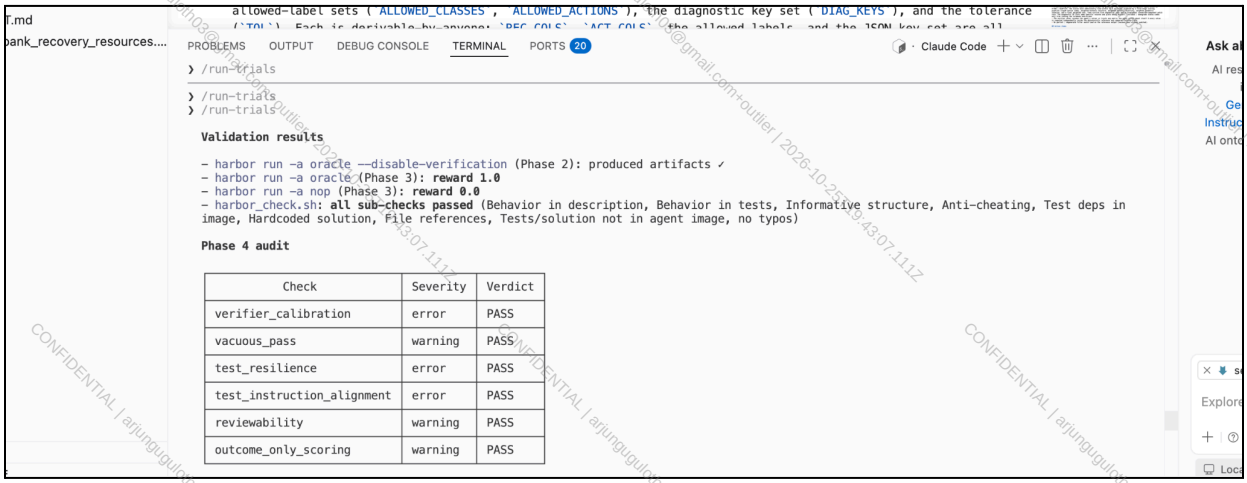
We recommend using Claude to identify where fixes are needed using this task. If they are doable for you to fix, please fix. Else, identify what needs to be fixed and return to the feedback step to provide to the attempter to fix.

1. Verify Oracle, NOP, and Harbor Checks

Confirm that Claude automatically ran the following checks and that they passed:

You will see something along the lines of:

- *harbor run -a oracle (Phase 3): reward 1.0*
- *harbor run -a nop (Phase 3): reward 0.0*



ONLY If you do not see confirmation that these checks were run automatically OR you want to rerun only oracle / nop after fixes, execute:

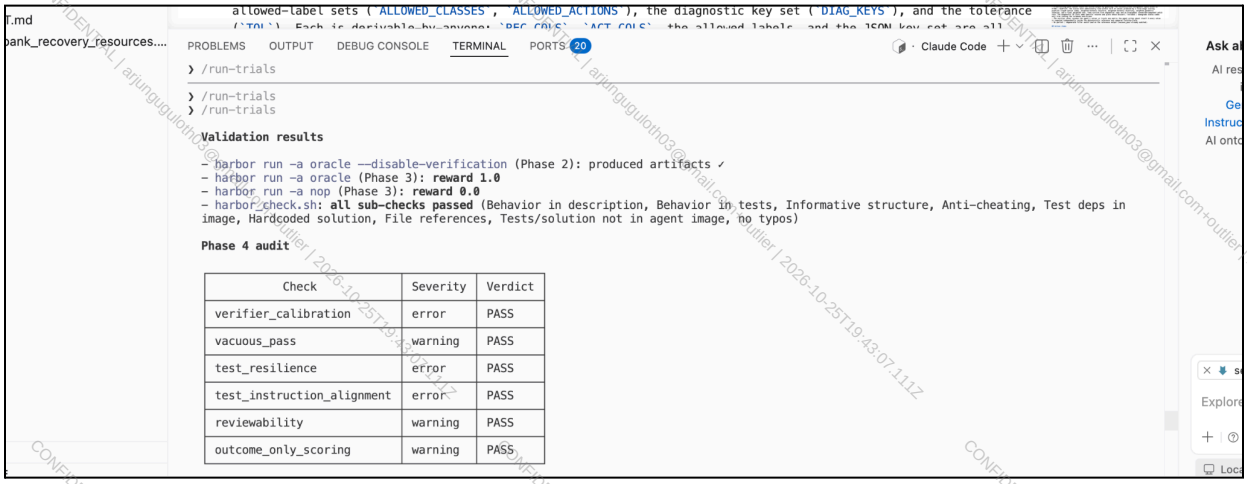
```
Shell
/run-oracle
/run-nop
```

2. Verify Verifier Checks

Confirm that Claude automatically ran verifier audits and that all checks passed.

You will see something along the lines of this table below:

- *harbor_check.sh: all sub-checks passed*



Verifier results may appear in the terminal output and the following file:

```
None
verification-reports/<task-name>.md
```

PLEASE GO THROUGH THESE VERIFICATION REPORTS TO CHECK VERIFIER ISSUES.

ONLY If verifier checks were not run automatically and you do not see any filled verification-reports file OR if you made fixes and want to rerun the verify test, execute:

```
Shell
/verify-task
```

If there are failures or warnings, determine whether the issue is caused by the verifier itself or by the task implementation before proceeding.

3. Run Agent Trials (required)

Execute:

```
Shell
/run-trials
```

Notes:

- **Agent trials are required for every L0 sandbox session before a task can be sent forward.**
  May take 30min-1.5 hours to complete → feel free to leave it running on your laptop.
- Provides a significantly more accurate difficulty assessment.
- Passing agent trials increases confidence that the task will pass downstream quality evaluations (the next quality evaluation is a similar agent trial).
- The three agents run are GPT 6.1 Sol, GPT 6 Sol, and Grok 4.7 (all highest reasoning). GPT 6 Sol is our difficulty proxy from L0 to L1 — it is the pass-forward bar.
  - The task is considered sufficiently difficult only if GPT 6 Sol fails at least 2 of its 3 trials AND a minimum of 5 of the 9 trials fail
  - If GPT 6 Sol passes more than 1/3 of trials, the task cannot advance regardless of how the other two agents performed. It must be sent back to the attempter for improvement on difficulty.
  - The "analyze" step provides additional insight.

4. Insights Checks (required)

Insights is a set of 22 checks that simulates the work of a Terminal Bench Reviewer and audits the entire task.

For technical issues, the skill will automatically fix any problems it detects. However, flags related to expert domain judgment require your input before a decision can be made.

Before submitting, make sure to review the final report and identify which flags require your attention. **Keep in mind that some flags may be invalid.** If you encounter an invalid flag, leave it as is and continue reviewing the remaining items before submitting the task.

Execute:

```
Shell
/run-insights
```

Notes:

- **May take 30min → feel free to leave it running on your laptop.**
- Auto-fixes safe/mechanical issues and re-verifies oracle (1.0) / nop (0.0) after each fix.
- Flags for your input anything that changes instruction meaning, tolerance/threshold values, verifier grading logic, oracle correctness, or trips a security signature, you decide whether each flag is valid and fix it.
- Writes a report under *jobs/insights-<task>-<ts>/* with a final **READY** / **NEEDS EXPERT** verdict; resolve open error-severity flags before submitting.

❗ Important clarification ❗

In a normal workflow, a reviewer will use just two commands:

1. **/build-task: Mandatory**. This command creates all task files and runs all checks. You have to run it, and them check whether all files were correctly created and that all checks pass.
   a. If the feedback flags issues, you have to decide whether the flags are valid or not. In the case the flags are valid, you have to fix the task/files.

2. **/run-trials: Mandatory.** This command runs 3 top AI agents that will try to solve the prompt. Your goal here is to analyze the results to determine whether the task is difficult enough and free from additional issues (specification issues, poorly calibrated verifiers, etc).
   a. This test is the best difficulty signal you could have.
3. **/run-insights: Mandatory.** Audits the finished task with the full insight-check suite, auto-fixes safe/mechanical issues, and flags anything needing your judgment. Review and resolve the flags before submitting.

5. Capture Files

Once you are satisfied with the quality and completeness of the generated files:

1. Return to the Outlier platform.
2. Click Capture Files.
3. Verify that files from the /tasks directory were captured successfully.

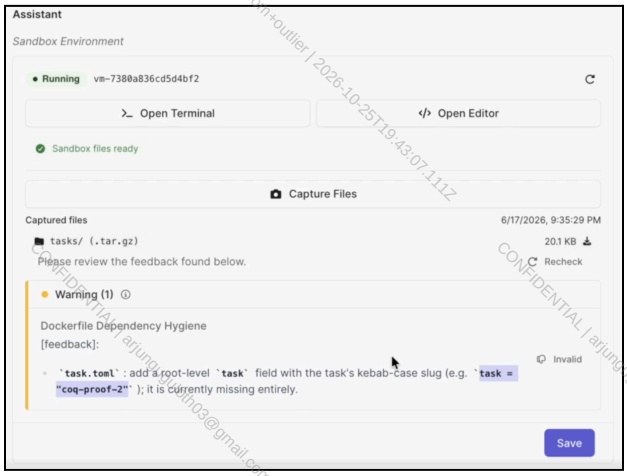


Important: Any changes made after capturing files require you to click Capture Files again before submitting.

6. Final Linter Checks

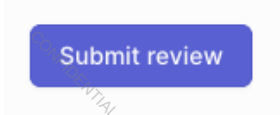
After clicking Save, Outlier will automatically run final linter checks.

- Review all linter feedback.
- Implement required fixes using Claude Code.
- Re-run validation if necessary.
- Click Capture Files again if any files were modified.



7. Submit

After all checks pass and files have been captured successfully, submit the task.



Note: If you run into sandbox or Claude issues, please reach out to a QM in the war room on OC. If it helps you identify that the original attempter had issues in the task design (e.g., difficulty), you should copy any feedback from Claude in your interactions, go back to the quality review section, paste the feedback/explain, and then change your score/fixes needed status = yes.

8. If a Reviewed Task Was SBQd to You

After your submission, an automated QC pipeline runs a battery of checks and computes a routing decision. A task is SBQd to a reviewer when the pipeline finds the task too easy (agents solved it too reliably without any cheating incentive) Your job is to read the report provided in the feedback section, confirm the flag is real (not a fluke or an artifact of the pipeline itself), and decide the next step: most often, that the task is genuinely too easy and needs to go back to the author. Other reasons include:

- **Cheat:** The model was able to exploit the verifiers and cheat to succeed on the task
- **Harbor nop:** The Nop agent should fail the task or score zero; if it scores high or successfully completes it, this usually flags an adversarial reward hack or a flaw in the testing environment's integrity
    - ➜ TLDR: ensures no answer is NOT rewarded
- **Harbor Oracle:** Runs an expected correct solution script in your Docker/Daytona environment to confirm that the task is solvable and that your setup, datasets, and scripts are working perfectly.
    - ➜ TLDR: ensures correct solution is rewarded

Reading the Auto QC Report

Each cell has three parts:

1. Score block. Overall Archie Score plus four component checks, each ✅/❌ x/5:
    - Harbor nop / Harbor Oracle: sanity checks that the task scaffold and reference solution behave correctly.
    - Harbor Pass@k CODEX: how many of the Runs (independent agent attempts, no cheating framing) came back Passed / Failed / Errored.
    - Harbor CHEAT (opus): whether an agent explicitly prompted/incentivized to exploit the grader succeeded (Cheat succeeded: True/False).
2. Routing Decision. The automated verdict, with a Reason(s) list when it's not a clean pass:
    - 🟢1: Advanced to next stage ➜ no blocking issues.
    - 🟡0: Sent back to reviewer ➜ blocked; read the reason(s) (most commonly "More than 1/3 of pass@k trials passed").
    - QM REVIEW NEEDED ➜ the pipeline itself couldn't render a verdict (e.g., pass@k runs and Errors out, or the cheat check errors or returns an unrecognized verdict).
    - ➜ *You will mostly see tasks with Routing Decision 0.*
3. "**--- Read More —**" links ➜ full Pass@k and CHEAT trajectories. ⚠️ CHEAT analyze checks block may appear above them with qualitative context (e.g., reward_hacking, task_specification). This is informational only and does not by itself drive routing.

When a task is "too easy"

A Routing Decision: 0, whose reason is pass@k trials passing above threshold, means multiple independent, non-cheating agent attempts solved the task. Note: this routing reason is driven specifically by GPT 6 Sol's pass rate exceeding 1/3 of trials. Confirm the GPT 6 Sol-specific result before deciding next steps — do not override this flag based on other models' fail rates, even if manual review suggested the task was difficult enough.

For this case:

1. Confirm it's genuine, not an artifact, before agreeing with the auto-flag:
    - Open the Pass@k trial link(s) and check that the passing runs actually solved the task legitimately (correct reasoning/output), not via a verifier loophole or lucky partial credit.

- Check the ⚠️ CHEAT (opus) analyze context (if present) for signs the legitimate runs benefited from the same defect the cheat run exploited (e.g., a leaked/weak verifier, an injected/contaminated instruction). If so, this is a task-integrity bug, not just a difficulty issue.
- If it's a clean, too-easy case (passes are legitimate, no artifact), add a note in the feedback field for the author, e.g., pointing at which pass@k run(s) show the shortcut being taken and a concrete lever to close it (remove a hint, add a distractor, deepen the reasoning chain, tighten the verifier). You can also share the trajectory links with contributors when reasonable, so that they can see how the models solved their tasks.

Testing Summary

#	Test	Command	What It Does	What It Means / Pass Criteria
1a	Oracle Check	/run-oracle	Runs the reference (correct) solution against the verifier.	Confirms the verifier accepts a known-good solution. Should receive a reward / score 100%.
1b	Nop Check	/run-nop	Runs an empty / no-op solution against the verifier.	Confirms the verifier doesn't pass trivially. Should NOT receive a reward. If it passes, the verifier is broken.
2	Verifier Checks	/verify-task	Runs the full verifier test suite (usually auto-runs on file creation). Results saved to verification-reports/<task-name>.md.	All results should be PASS. Any FAIL needs investigation with Claude.
3	Difficulty Check	/terminal-bench-coder evaluate difficulty	Claude analyzes the task against in-skill examples and judges whether it's sufficiently difficult.	Tends to under-flag — if it says "easy," it almost certainly is easy. If it says "hard," it's likely (but not guaranteed) hard. Use pass@ for a more accurate read.

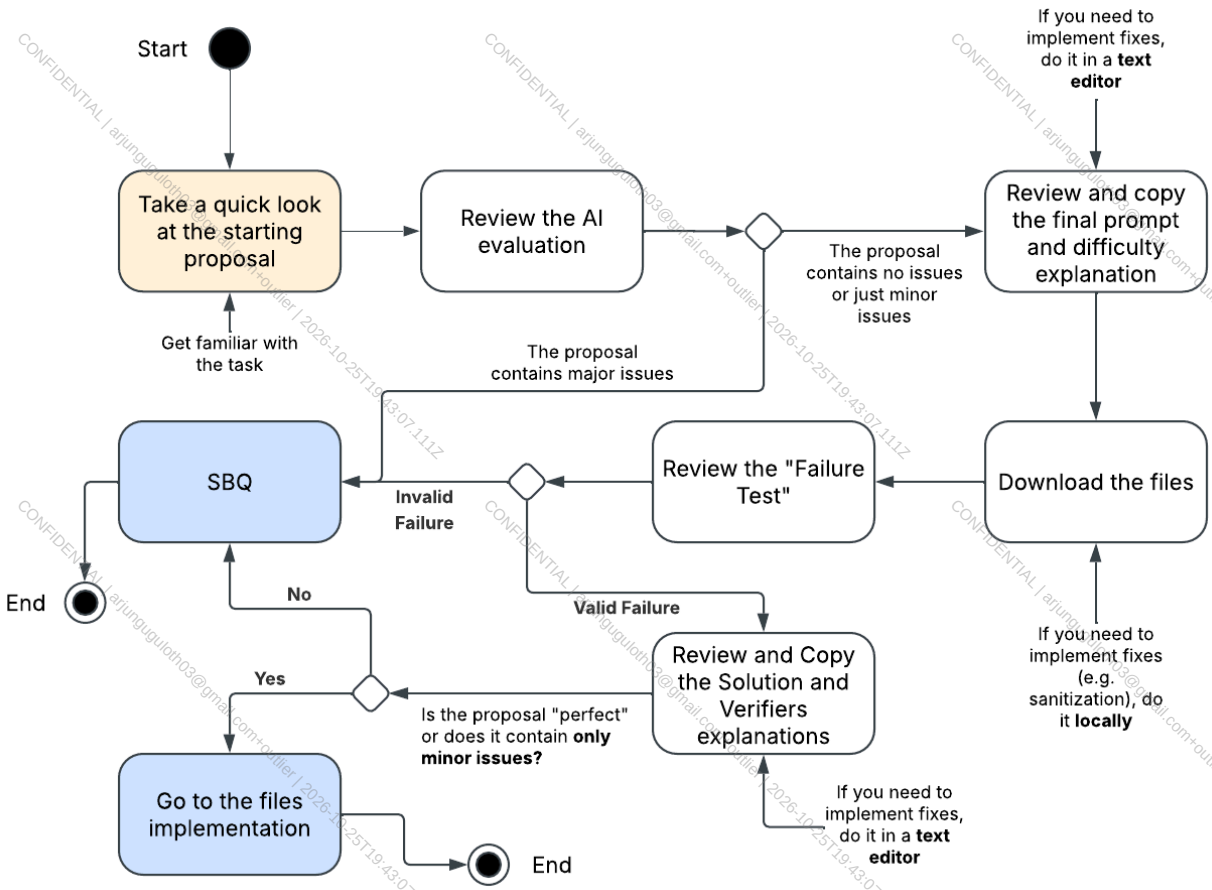
4	Agent Trials (pass@) — *required*	/run-trials	Runs 3 agents in parallel against the prompt (~1 hr). Mirrors the official quality-incentive check.	Task is considered sufficiently difficult only if GPT 6 Sol fails at least 2 of 3 trials AND a minimum of 5 of the 9 trials fail. If GPT 6 Sol passes more than 1/3, the task fails this check regardless of other agents' results.
5	Insights	/run-insights	Audits the finished task with the full insight-check suite, auto-fixes safe/mechanical issues, and flags anything needing your judgment.	Task needs to have all insights checks resolved or be disputed on all checks to be fully passing to final layer. Write disputes in the “Notes for Coder” box.
6	Capture Files	*(UI action)*	Top-left ☰ icon → File → Save, then "Capture Files" → "Save" in Outlier.	Stores generated files in a downloadable format. Must re-capture after ANY file change.
7	Linters	*(automatic on Save)*	Runs file-formatting and task-quality checks.	For file hygiene: paste feedback into Claude in the sandbox to fix, then re-Capture. For task quality (difficulty, verifiability, solution quality): copy feedback BEFORE selecting the "needs fixes" field (selecting it wipes sandbox files), then route to the attempter.

Reviewer Workflow Summary

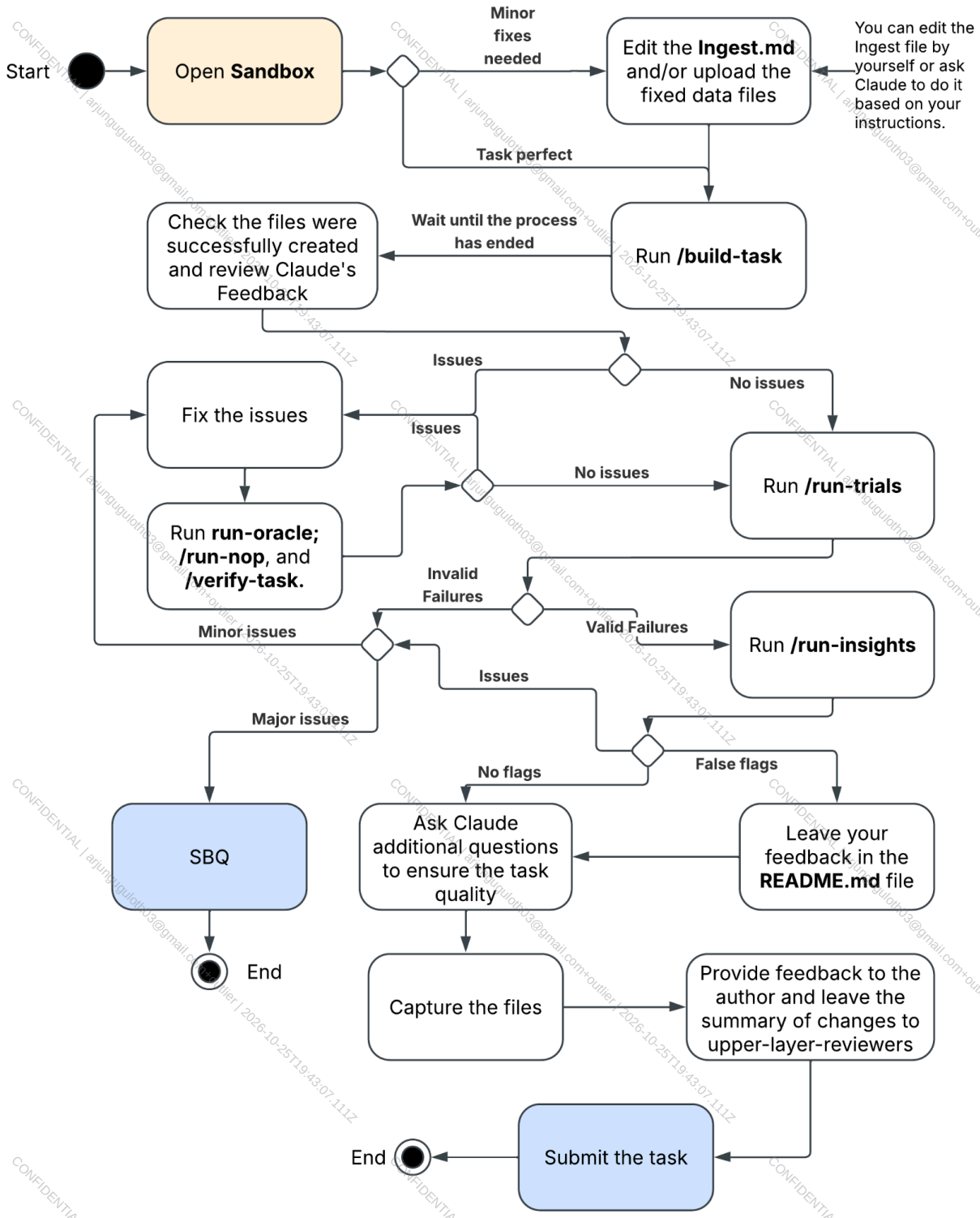
The flow charts below summarize your workflow as a reviewer for reviewing a task from scratch.

Workflow in the task:

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z



Workflow in the Sandbox for implementing the files:


Start
Open Sandbox
Minor fixes needed
Edit the Ingest.md and/or upload the fixed data files
You can edit the Ingest file by yourself or ask Claude to do it based on your instructions.
Task perfect
Run /build-task
Wait until the process has ended
Check the files were successfully created and review Claude's Feedback
Issues
No issues
Fix the issues
Issues
No issues
Run /run-trials
Run run-oracle; /run-nop, and /verify-task.
Invalid Failures
Valid Failures
Run /run-insights
Minor issues
Issues
Major issues
False flags
No flags
SBQ
Ask Claude additional questions to ensure the task quality
Leave your feedback in the README.md file
End
Capture the files
Provide feedback to the author and leave the summary of changes to upper-layer-reviewers
End
Submit the task

Payment Guidelines

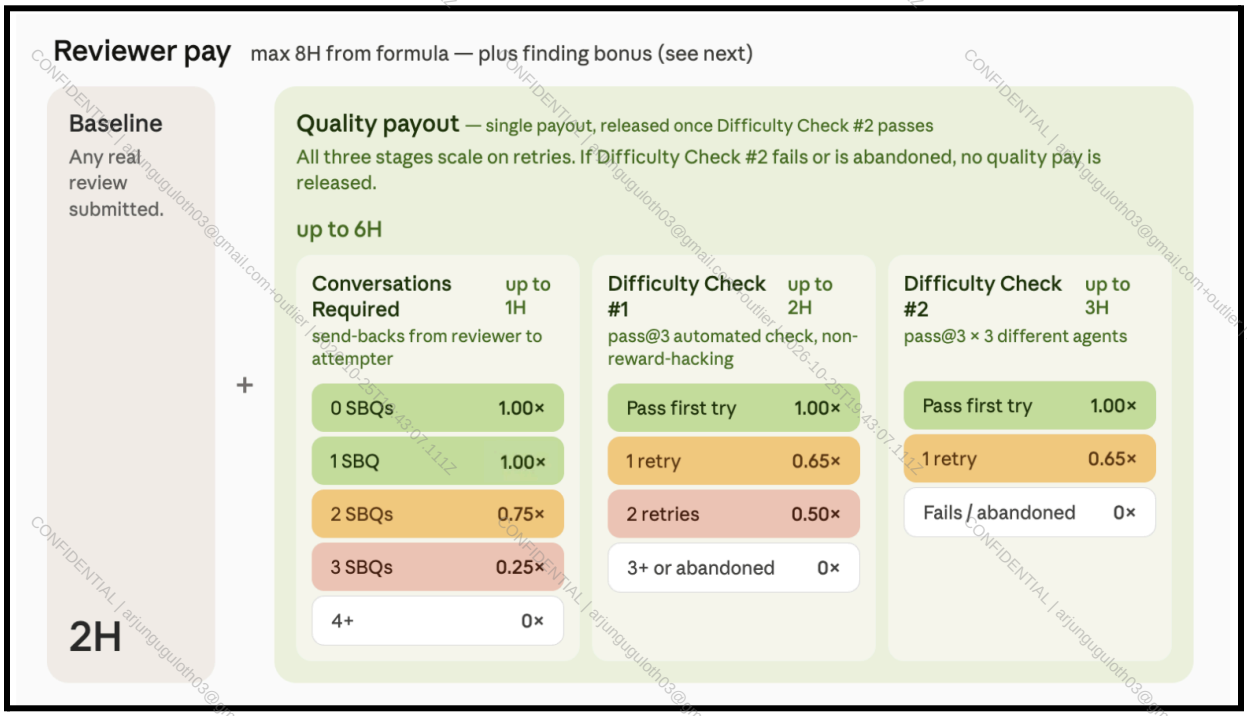
💰Pay is **Per Task.**

Quality and **Difficulty** are the most important stats in the project.

You will receive 1-3 separate payments for each task depending on the quality and difficulty of your task. All pay is based on your Expert Skill Rate.

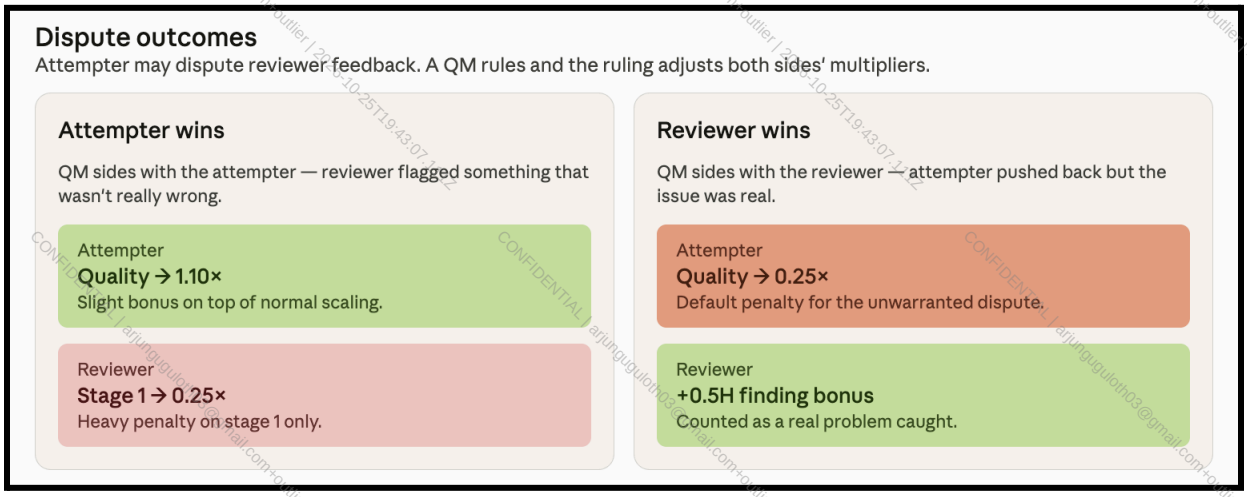
1. Base pay – 2 hours
    a. You will receive this for every task upon **submission**
2. Send Backs - up to 1 hour
    a. You will receive this based on how many times the task has to be SBQ'd. **Be sure to leave thorough, helpful feedback for the attempter** if you are sending it back. Ideally, a task is SBQ'd 0-1 times.
3. ⭐Quality/Difficulty – up to 2 hours
    a. You will receive this depending on **the pass@3 check + harbor analyze checks that run.**
4. ✨Quality / Difficulty – up to 3 hours
    a. You will receive this based on a set evaluation of how many fail out of 9 model runs attempting the task
    b. ‼️Difficulty is the most important factor of this project. Easy tasks will not receive any reward (0 hours)
        i. To get the full reward for this aspect, refer back to the "Difficulty" sections of the guidelines to make sure the task is properly difficult. If giving feedback, it may be helpful to refer back to this section and identify actions the attempter could take to increase difficulty.

CONFIDENTIAL | arjungugulothu03@gmail.com+outlier | 2026-10-25T19:43:07.111Z



Note: To maintain high quality tasks, you should discuss with the attempter in the automatic Outlier Community Topic created when you claimed the task. If you SBQ a task that you and the attempter did not reach an agreement on after discussion, the attempter can dispute, and the following situation will apply:

1. <u>The task you SBQ'd was actually perfect (no valid issues that the reviewer (you) flagged, nor any others visible)</u>
    a. You will receive a penalty of 0.25x for that task and the attempter receives a slight bonus
2. <u>The task indeed had an issue.</u>
    a. Your quality incentive will receive a slight bonus, and the attempter penalized



FAQs

1. How much time do I have to review?
    a. You have 8 hours of claim time + can pause for 16 additional hours
2. What happens if I need to make changes to the task when I'm in the sandbox?
    a. If you are going to move the task forward (aka not SBQ), then you are allowed to make changes in the sandbox and **are not required to make those same changes in the previous steps to save time.**

CONFIDENTIAL | arjungugulothO3@gmail.com+outlier | 2026-10-25T19:43:07.111Z